

Personalized Recommendation Agents with Self-Consistency

Zijing Wu
University of Science and Technology
of China
Hefei, China
edenwu@mail.ustc.edu.cn

Leheng Sheng*
National University of Singapore
Singapore
leheng.sheng@u.nus.edu

Yuanlin Xia
Renmin University of China
Beijing, China
xiayl@ruc.edu.cn

Yi Zhang
University of Science and Technology
of China
Hefei, China
zy1230@mail.ustc.edu.cn

Yuxin Chen
National University of Singapore
Singapore
yuxin.chen@u.nus.edu

An Zhang
National University of Singapore
Singapore
anzhang@u.nus.edu

Abstract

This paper presents the 2nd place solution of the WWW'25 AgentSociety Challenge (Recommendation Track), which aims to advance the development of Large Language Model (LLM)-based agents for personalized recommendation. During this challenge, participants are tasked with building LLM-based agents capable of ranking items according to user preferences. To evaluate the practical capabilities of these agents, the organizer provides a robust evaluation platform that faithfully simulates real-world recommendation scenarios across multiple domains (e.g., Yelp, Goodreads, and Amazon).

To tackle this challenge, our team, RecHackers, develops an agent-voting system inspired by the idea that self-consistency across multiple LLM outputs can enhance the accuracy. Specifically, our approach follows a *prompting-sampling-voting* paradigm. We first carefully design prompts for individual agents based on user interaction and item characteristics. Then, we query LLM-based agents multiple times with a controlled temperature to obtain a diverse range of candidate rankings. Lastly, we aggregate these rankings with predefined voting mechanisms, such as majority voting or Borda count, to derive the final ranking list. This simple yet effective solution achieved the 2nd place in the WWW'25 AgentSociety Challenge (Recommendation Track).

CCS Concepts

• Information systems → Information retrieval.

Keywords

Large Language Models, Recommender System, LLM-based Agents

ACM Reference Format:

Zijing Wu, Leheng Sheng*, Yuanlin Xia, Yi Zhang, Yuxin Chen, and An Zhang. 2025. Personalized Recommendation Agents with Self-Consistency. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3701716.3719229>

*Leheng Sheng is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW Companion '25*, Sydney, NSW, Australia
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1331-6/2025/04
<https://doi.org/10.1145/3701716.3719229>

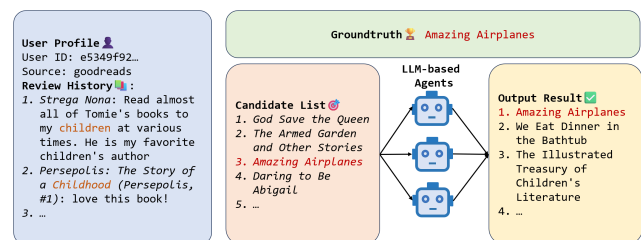


Figure 1: Personalized book recommendation example. Given a user with a history of reviewing children's books, the LLM-based agents recommend *Amazing Airplanes*, a children's picture book.

1 Introduction

The remarkable advancement of large language models (LLMs) in recent years has paved the way for a new research direction: LLM-based agents. Adopting LLMs as the brain and equipped with memory, planning, and tool using modules, these agents have achieved great success in various domains, such as society simulation [8, 16], software development [9, 10], and embodied robotics [15]. Attracted by the wide success of LLM-based agents, recent efforts have also begun to explore deploying them in the field of recommender system. Benefiting from their human-like behaviors and user intention understanding capabilities, these LLM-based agents show great potential in acting as recommendation assistants [6, 11, 14] and user simulators [12, 16]. Despite these initial explorations and potentials, the real-world deployment of LLM-based agents in the recommender system still remains largely unexplored, primarily due to the scarcity of publicly available large-scale datasets and the high costs associated with extensive experimentation.

To bridge this gap, WWW'2025 organized the AgentSociety Challenge (Recommendation Track), aiming to advance research and promote practical solutions that leverage LLM-based agents in real-world recommender systems. The recommendation track of this challenge specifically focuses on the problem of generating personalized item rankings based on user interactions (see the illustrative example in Figure 1). Participating teams are required to design LLM-based recommendation agents that can effectively model user preferences and deliver tailored item suggestions.

In response to this challenge, our team, RecHackers, proposes a simple yet effective agent-voting system for precise user preference modeling and accurate item ranking. The core of our framework follows a *prompting-sampling-voting* paradigm, inspired by the observation that self-consistency in LLMs can significantly enhance accuracy [7, 13]. Firstly, we carefully design the prompt for individual agents to generate personalized item rankings by incorporating user history and item information. After that, we prompt LLM-based agents multiple times under a controlled temperature to sample a diverse set of candidate rankings. Finally, we aggregate these diverse candidate rankings through pre-defined voting strategies (*i.e.*, majority voting [13] or Borda count [3]) to deliver a final ranking. With such a *prompting-sampling-voting* paradigm, our approach effectively ensures the rank accuracy by considering the agent consistency. As a result, this simple yet effective approach yields prominent performance on both simulated and real-world user data, achieving 2nd place in the WWW'25 AgentSociety Challenge (Recommendation Track).

The remainder of this paper is organized as follows. We first introduce the recommendation track of the AgentSociety Challenge in Section 2. Next, in Section 3, we provide an in-depth description of our multi-agent voting framework. After that, we present a comprehensive evaluation of our system in Section 4, including a comparison with existing baselines, an ablation study, and further analyses. In the end, we conclude our findings in Section 5.

2 WWW'25 AgentSociety Challenge (Recommendation Track)

In this section, we comprehensively review the WWW'25 AgentSociety Challenge (Recommendation Track). We first introduce the task of the recommendation track in Section 2.1. After that, we introduce the evaluation details of this task in Section 2.2.

2.1 Recommendation Track

In this track, participants are tasked with developing LLM-based agents that can provide personalized recommendations based on the user's historical preferences. Specifically, given an item list that contains a target item with which the user will interact, these LLM-based agents need to rank the item list in a way that places the target item as close to the front as possible [5].

2.2 Track Evaluation

The evaluation of participants' recommendation agents is based on their capacity to accurately rank items that users are most likely to select from a given list of candidates.

There are two evaluation phases in the competition: the development phase and the final phase. In the development phase, participants have access to the mixed datasets (Section 2.2.2), whose ground truth is generated by an official LLM-based agent's simulation. Participants are encouraged to use these offline datasets to refine their agents and submit them online for selection into the final phase, where the top 20 teams will be chosen. In the final phase, the online dataset is updated with 40% simulation data and 60% real data. The datasets will be used to evaluate the final performance of the agents submitted by the participants.

2.2.1 Rules. Participants are required to design LLM agentic workflows within a predefined modular design space, ensuring efficient module selection and effective performance. The final evaluation contains a total of 600 tasks (*i.e.*, users), with a maximum parallelism of 5 and a strict time limit of 120 minutes¹. Given this constraint, participants must carefully control the computational complexity within their agents to ensure efficient recommendation processing.

2.2.2 Datasets. The datasets used in this challenge are sourced from three online platforms as follows:

- **Yelp**²[1]: Yelp is derived from a U.S. business review website that covers specific domains such as restaurants, shopping centers, and hotels. The platform has collected data on over 150,000 businesses in 11 metropolitan areas, as well as 6.9 million reviews.
- **Amazon**³[4]: Amazon originates from the Amazon shopping platform, encompassing 571.54 million reviews from 1996 to 2023. In addition to user ratings and reviews of products, it also involves product details such as price, brand, and category.
- **Goodreads**⁴: Goodreads data is selected from a book social platform. 876,145 users give over 228 million user-book interactions for 2.36 million books. Goodreads also includes author information and meta data of the books.

In evaluation, the specific datasets utilized in this track have two branches: **offline** and **online**. The offline ground truth is generated by a predefined LLM-based agent simulating user choices based on context and task queries. The final phase of online data is composed of 40% simulation data and 60% real data.

2.2.3 Metrics. The commonly used metric Top- N Hit Rate ($\text{HR@}N$) is employed for evaluating the recommendation performance of LLM-based agents. The $\text{HR@}N$ metric measures how often the ground truth item appears in the top N of the ranked list of 20 candidate items, which can be formulated as follows:

$$\text{HR@}N = \frac{1}{T} \sum \mathbb{1}(p_t \in P_t^N), \quad N \in \{1, 3, 5\},$$

where T is the total number of test tasks, and $\mathbb{1}(\cdot)$ is an indicator function that equals 1 if the ground truth p_t appears in the top N recommendations P_t^N within the ranked list P_t , and 0 otherwise.

3 Methodology

Our approach follows a simple yet effective *prompting-sampling-voting* paradigm, drawing inspiration from the success of self-consistency in LLMs [13]. Generally, we sample multiple ranking lists with a fixed temperature and aggregate them into the final ranking list via a voting mechanism to deliver accurate recommendations. We illustrate the pipeline of our method in Figure 2. In the following sections, we provide the details of prompt design, response sampling, and voting respectively.

3.1 Prompt Design

We equip each agent with a carefully designed prompt to generate the personalized ranking list. As shown in Figure 3, our prompt

¹<https://tsinghua-fib-lab.github.io/AgentSocietyChallenge/pages/submission-guidelines.html>

²<https://www.yelp.com/dataset>

³<https://amazon-reviews-2023.github.io/>

⁴<https://sites.google.com/eng.ucsd.edu/ucsdbookgraph/home>

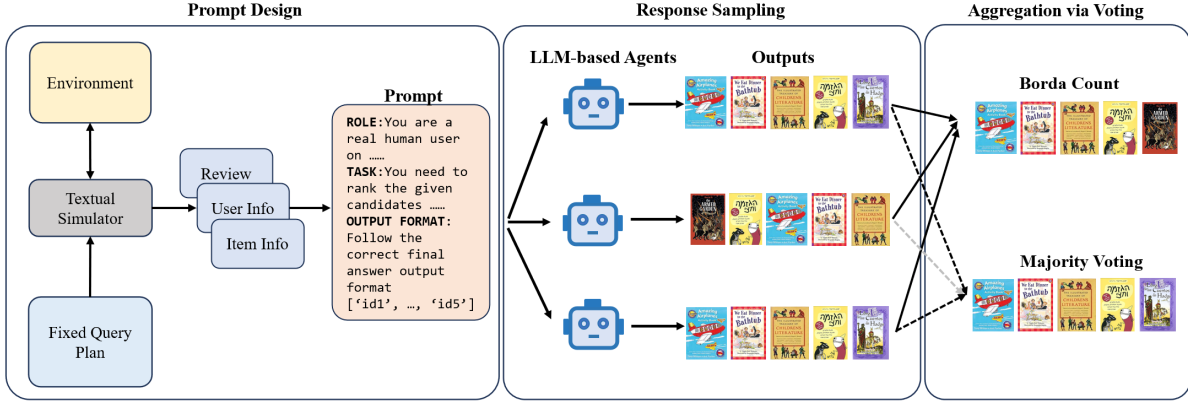


Figure 2: Overview of our prompting-sampling-voting paradigm. In the prompt design phase, we apply a fixed query plan to extract information from the text simulated environment to construct the prompt. In the sampling phase, we call the LLM-based agents to get a set of top-5 items. In the voting phase, we apply two aggregation methods: Borda count and majority voting to get the final answer.

Prompt Template

ROLE:

You are a real human user on {dataset}, a platform for crowd-sourced {item_type} reviews. Here is your profile {user} and review history {review_history}

TASK:

You need to rank the given candidates {candidate_list}, based on their information {item_info}. Candidates that closely match your review history and have higher ratings should be ranked higher.

OUTPUT FORMAT

Your final output should be ONLY a ranked item list of {candidate_list}. You only need to select the top 5 items from the candidate list.

DO NOT output your analysis process!

Follow the correct final answer output format STRICTLY, ['id1', 'id2', 'id3', 'id4', 'id5']

Figure 3: Prompt template for ranking candidates.

consists of three crucial components: role, task, and output format. Here, we use the angle bracket (i.e., {placeholder}) to denote a placeholder that needs to be filled with specific information retrieved through the interface *textual simulator* provided by the competition organizer. In the role section, we define the responsibilities of the agent. Next, we require the agent to rank the candidate item list based on the user’s historical information. Additionally, we restrict the output format to prevent generating non-compliant item lists. For retrieving the required user and item information for placeholders, we employ a fixed planning module to query the textual simulator, which is the same as the baseline.

3.2 Response Sampling

We prompt the LLM-based agent ϕ multiple times with a high temperature τ to sample diverse ranking lists, which can be formulated as $P_i = \phi_\tau(q)$. Here q represents the prompt that has been filled by the retrieved information, ϕ denotes the LLM-based agent, and i denotes the i -th sampling time.

Specifically, by sampling n times, we can get a set of responses $\mathcal{P} = \{P_1, \dots, P_n\}$. According to the output format specified in the prompt, we expect P_i to be a list consisting of five item IDs (i.e., $P_i = [p_i^{(1)}, \dots, p_i^{(5)}]$). If the response does not match this format, we simply discard this response P_i and set P_i to a null list.

3.3 Aggregation via Voting

After sampling n diverse ranking lists, we adopt a voting mechanism to aggregate them into the final ranking result A . This voting mechanism can make the results more accurate by considering the self-consistency between agents [7, 13]. Specifically, we attempt two voting methods: majority voting [13] and Borda count [3].

Majority voting [13] is a simple approach that selects the most frequently appeared items in each response as the answer A , which can be formulated as:

$$A = \arg \max_{P \in \mathcal{P}} \sum_{i=1}^n \mathbb{1}\{P_i = P\},$$

where $\mathbb{1}\{\cdot\}$ is the indicator function that equals 1 if the condition is true, and 0 otherwise. This aggregation method is similar to the CoT-SC [13] without an explicit thinking process. Adopting such a voting mechanism, the final ranking list contains items that appear the most frequently among all the responses.

Borda count [3] is a more fine-grained voting method that considers the ranking positions of items in each candidate responses. Specifically, we define a ranking-based score function to measure the importance of each item, and generate the final ranking list by sorting items according to their scores. The score function is

defined as follows:

$$g(p) = \sum_{i=1}^n \sum_{j=1}^5 (6-j) \mathbb{1}\{p_i^{(j)} = p\},$$

which assigns higher scores to higher-ranked items. For items that do not appear in the top-5 positions, we set their scores to 0. After calculating the scores of all the items that appear in the responses, we re-rank them $\mathcal{P}$ based on their g values, and select the top-5 items with the highest g values as the final answer A . We first evaluate these two voting mechanisms on the offline data. We empirically find out that Borda count consistently outperforms the majority voting (see Table 1). Therefore, we adopt the Borda count for our submissions in the final phase of this challenge.

4 Results

We conduct comprehensive experiments on both offline and online data to evaluate the effectiveness of our proposed method. We first elaborate on the implementation details in Section 4.1. After that, we present the overall performance of our proposed method on both online and offline data in Section 4.2. Then, we give details about how we select the hyperparameters (*i.e.*, the number of agents n and temperature τ) in Section 4.3.

4.1 Experimental Settings

4.1.1 Implement detail. We present the implementation details of our experiments in this section, to ensure reproducibility and transparency of our work. For the baseline, we use the official code⁵ provided by the competition organizer, with a temperature set as $\tau = 0.1$ by default. For the final submission of our proposed method, we employ the prompt design and sampling strategy as described in Section 3, with the number of agents set as $n = 3$ and the temperature set as $\tau = 1.0$.

4.1.2 Evaluation Details. We evaluate our method on both offline and online data. For offline evaluation, we use the gpt-4o-mini model from OpenAI⁶ as the API backend for agents. For online evaluation, the competition organizer provides the Qwen-72B-Instruct model [2] as the API backend for agents. For both offline and online evaluations, we report the Hit Rate (HR@ N) metric, which measures the proportion of relevant items found within the top- N recommendations. Specifically, we report the Hit Rate with N set as 1, 3, and 5 (*i.e.*, HR@1, HR@3, and HR@5).

4.2 Overall Performance

We adopt both majority voting and Borda count for the offline experiments, while only choosing the Borda count for the online evaluation. We demonstrate the superior performance achieved by our simple yet effective agent-voting system in the Table 1. We summarize key observations as follows:

- **On the offline data, both the Borda count and majority voting methods significantly outperform the baseline.** Additionally, the Borda count, which assigns greater weight to higher-ranked items, consistently outperforms majority voting across

all metrics. This underscores the importance of fully leveraging ranking position information to achieve more accurate and effective ranking results.

- **Our method exhibits excellent performance on the online data, achieving 2nd place among all the teams.** Additionally, it achieves the best performance on both HR@3 and HR@5 metrics. A possible explanation might be that Borda count ensures that as long as an item is ranked sufficiently high in a single LLM-based agent's output, it remains influential regardless of whether it appears in other responses. This promotes diversity in the final ranked list, allowing more than five distinct items to appear in the responses $\mathcal{P}$, which in turn contributes to consistently high top-5 and top-3 hit rates.

4.3 The Effect of Hyperparameters

We select the optimal hyperparameter for online evaluation by tuning the hyperparameters on the offline data. In this section, we present the hyperparameter study (*i.e.*, number of agents n and temperature τ) of our method.

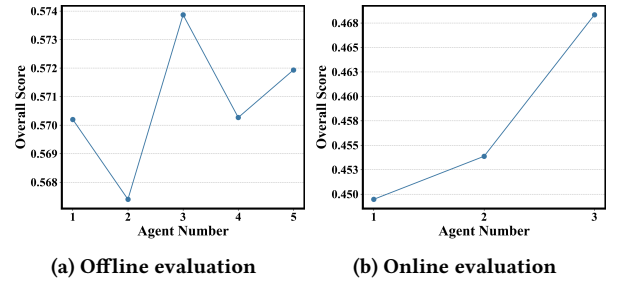


Figure 4: Effect of the number of agents on the overall score with temperature set as $\tau = 1.0$. (4a) Offline performance with agent number ranging from 1 to 5. (4b) Online performance with agent number ranging from 1 to 3.

4.3.1 Number of agents n . The number of agents n in the voting phase presents a trade-off between performance and computational cost, as illustrated in Figure 4. In the online evaluation, performance improves from 0.449 to 0.468 as the agent number increases from 1 to 3. The overall score improves demonstrating the benefits of ensembling multiple LLM-based agents. Due to computational constraints in the competition, we are unable to explore the higher number of agents in this setting. In the offline setting, we observe a similar trend, with performance peaking at $n = 3$ before slightly declining. This suggests that for this recommendation task, three agents are sufficient to capture a diverse range of user preferences and item characteristics.

4.3.2 Temperature τ . In our sampling process, the temperature parameter governs the diversity of the LLM-based agent output during the sampling phase. Lower temperatures lead to more deterministic and focused generations, while higher temperatures promote diversity but can also introduce more noise.

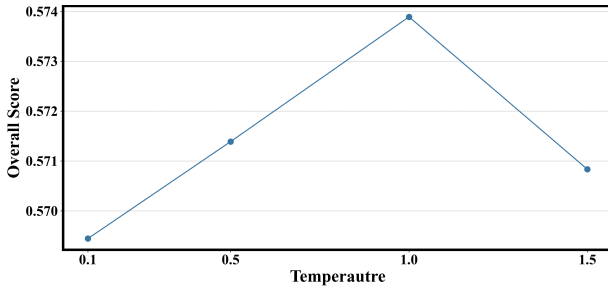
As shown in Figure 5, we observe a trend of increasing and then decreasing performance as the temperature rises on offline evaluation. This suggests that a moderate level of randomness, achieved with a temperature around 1.0 in our experiments, strikes the best

⁵<https://tsinghua-fib-lab.github.io/AgentSocietyChallenge/pages/overview.html>

⁶<https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>

Table 1: Recommendation metric of the methods. Bold and underlined indicate the best and the second-best performance, respectively.

Evaluation	Method/Team	HR@1	HR@3	HR@5	Score@Sim	Score@Real	Overall Score
Offline	Baseline	0.3258	0.5803	0.7091	-	-	0.5386
	Ours (majority voting)	<u>0.3667</u>	<u>0.6225</u>	<u>0.7200</u>	-	-	<u>0.5697</u>
	Ours (Borda count)	0.3708	0.6258	0.7250	-	-	0.5739
	Imp.% over the baseline	13.81%	7.84%	2.24%	-	-	6.59%
Online	baseline666	0.3050	0.4900	0.6283	0.6722	0.3426	0.4744
	RecHackers	0.2667	0.5033	0.6350	0.6417	0.3528	0.4683
	Dummyagent	<u>0.2983</u>	0.4783	0.6250	0.6250	0.3620	0.4672
	pp	0.2550	<u>0.4983</u>	<u>0.6333</u>	<u>0.6431</u>	0.3417	0.4622

**Figure 5: Effect of the temperature on Borda count in offline evaluation**

balance between exploration and exploitation. While some stochasticity is beneficial for exploring diverse recommendations, excessive randomness can lead to noise that hinders the effectiveness of the voting mechanism.

5 Conclusion

The WWW'25 AgentSociety Challenge (Recommendation Track), which focused on generating accurate and personalized item rankings, offered a challenging yet insightful competition setting for building personalized recommendation agents. To tackle this challenge, our team, RecHackers, developed an agent-voting framework that ensures the self-consistency between agents. Specifically, we carefully designed prompts for individual agents to generate personalized rankings, sampled diverse candidate rankings through multiple LLM queries, and aggregated them using predefined voting strategies to ensure robust and accurate final rankings. To enhance performance, we explored different prompt designs and voting strategies, balancing ranking accuracy, diversity, and stability. Additionally, we conducted ablation studies to assess the impact of different strategies in our framework. This simple yet effective approach demonstrated a strong performance, securing 2nd place in the WWW'25 AgentSociety Challenge (Recommendation Track). Beyond the competition, our findings highlighted the effectiveness of multi-agent collaboration and voting mechanisms in LLM-based recommender agents. We hope that our approach inspires further research into enhancing LLM-based recommender systems for real-world applications.

Acknowledgments

This research is supported by the NExT Research Center.

References

- [1] Nabihha Asghar. 2016. Yelp Dataset Challenge: Review Rating Prediction. *CoRR* abs/1605.05362 (2016). arXiv:1605.05362 <http://arxiv.org/abs/1605.05362>
- [2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. *CoRR* abs/2309.16609 (2023).
- [3] Peter Emerson. 2013. The original Borda count and partial voting. *Soc. Choice Welf.* 40, 2 (2013), 353–358.
- [4] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian J. McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. *CoRR* abs/2403.03952 (2024). <https://doi.org/10.48550/ARXIV.2403.03952> arXiv:2403.03952
- [5] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian J. McAuley, and Wayne Xin Zhao. 2024. Large Language Models are Zero-Shot Rankers for Recommender Systems. In *ECIR*.
- [6] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations. *CoRR* abs/2308.16505 (2023).
- [7] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. 2024. More Agents Is All You Need. *CoRR* (2024).
- [8] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *UIST*.
- [9] Chen Qian, Jiahao Li, Yufan Dang, Wei Liu, Yifei Wang, Zihao Xie, Weize Chen, Cheng Yang, Yingli Zhang, Zhiyuan Liu, and Maosong Sun. 2024. Iterative Experience Refinement of Software-Developing Agents. *CoRR* abs/2405.04219 (2024).
- [10] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *ACL*.
- [11] Yubo Shu, Haonan Zhang, Hansu Gu, Peng Zhang, Tun Lu, Dongsheng Li, and Ning Gu. 2024. RAH! RecSys-Assistant-Human: A Human-Centered Recommendation Framework With LLM Agents. *IEEE Transactions on Computational Social Systems* (2024).
- [12] Lei Wang, Jingsen Zhang, Hao Yang, Zhi-Yuan Chen, Jiakai Tang, Zeyu Zhang, Xu Chen, Yankai Lin, Hao Sun, Ruihua Song, et al. 2024. User Behavior Simulation with Large Language Model-based Agents for Recommender Systems. *TOIS* (2024).
- [13] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *ICLR*.
- [14] Yancheng Wang, Ziyang Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojian Huang, Yanbin Lu, and Yingzhen Yang. 2023. RecMind: Large Language Model Powered Agent For Recommendation. *CoRR* abs/2308.14296 (2023).
- [15] Zihao Wang, Shaofei Cai, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023. Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents. *CoRR* abs/2302.01560 (2023).
- [16] An Zhang, Yuxin Chen, Leheng Sheng, Xiang Wang, and Tat-Seng Chua. 2024. On Generative Agents in Recommendation. In *SIGIR*.